



CENTRO UNIVERSITÁRIO DE BRASÍLIA
CIÊNCIA DA COMPUTAÇÃO

Tomás Peva (RA: 22401580)
Rodrigo Maya Mesquita (RA:22402933)
Davi Santiago (RA: 22401141)
Gabriel Lopes (RA: 22400969)
Ronald dos Santos (RA:22403046)

SAPA – SISTEMA DE AUTOCONHECIMENTO PSICOLÓGICO AUTOMATIZADO

BRASÍLIA
2026



SISTEMA DE ANÁLISE DO PERFIL PSICOLÓGICO E ACADÊMICO GUIA TÉCNICO DE EXECUÇÃO E MANIPULAÇÃO DE DADOS (ETL)

Versão: 2.0

Repositório: https://github.com/Ronald-Jesus/Projeto_Integrador_1

1. Visão Geral da Arquitetura

O Projeto SAPA utiliza uma arquitetura de dados modular e resiliente construída em Python para coleta, processamento, consolidação e visualização de métricas relacionadas à saúde mental. O pipeline de dados (ETL) automatiza o cruzamento de microdados comportamentais individuais com macrodados em escala populacional, eliminando processamentos manuais.

O fluxo de dados segue as seguintes etapas:

1. **Extração (E):** Scripts autônomos acessam APIs REST (OMS, Kaggle), manipulam arquivos hospedados em nuvem (S3 AWS do SUS) e acessam servidores FTP governamentais (IBGE, DATASUS).
2. **Transformação (T):** Os dados brutos ("Dirty Data") são limpos em memória RAM utilizando a biblioteca pandas. Valores nulos são expurgados, text-mining é aplicado em diários textuais e anomalias dimensionais são filtradas cirurgicamente (foco em Transtornos Depressivos e Ansiedade).
3. **Carga (L) e Visualização:** Os artefatos são salvos temporariamente em CSV (data/processed/csv) e posteriormente consolidados em um banco de dados relacional (SQLite), planilhas de BI (Excel) e Dashboards visuais automáticos (Matplotlib/Seaborn).

2. Pré-requisitos de Ambiente (Local)

Para executar ou modificar os scripts de manipulação de dados localmente, o ambiente de desenvolvimento deve atender aos seguintes requisitos:

- **Linguagem:** Python 3.10 ou superior.
- **Gerenciador de Pacotes:** pip.
- **Bibliotecas Necessárias:** Bash pip install pandas requests openpyxl matplotlib seaborn pysus

3. Execução dos Scripts de Manipulação (ETL)

O sistema opera com 8 scripts independentes que garantem a extração isolada e segura de cada fonte de dados.

3.1. ETL 01: Hábitos e Emoções Diárias (FitLife)

- **Arquivo:** src/scripts/etl_01_kaggle_fitlife.py
- **Comando:** python src/scripts/etl_01_kaggle_fitlife.py

- **Função:** Extrai o *FitLife Emotions Dataset* via API do Kaggle, transformando hábitos diários (sono, batimentos cardíacos, calorias) e estados emocionais em numéricos.
- **Saída:** csv/emocoes_atividades.csv

3.2. ETL 02: Processamento de Linguagem Natural (Diários)

- **Arquivo:** src/scripts/etl_02_kaggle_journal.py
- **Função:** Acessa diários textuais e mapeia rótulos de processamento de linguagem natural (NLP) para determinar a emoção primária do usuário por dia. • **Saída:** csv/diários_emocoes.csv

3.3. ETL 03: Despesas Familiares (IBGE POF)

- **Arquivo:** src/scripts/etl_03_ibge_pof.py
- **Função:** Consome a Pesquisa de Orçamentos Familiares, extraíndo o custo financeiro "do bolso" dos brasileiros com medicamentos e tratamentos psiquiátricos, segmentado por renda e UF.
- **Saída:** csv/despesas_saude_pof.csv

3.4. ETL 04: Crise Hospitalar (DATASUS/SIH)

- **Arquivo:** src/scripts/etl_04_datasus.py
- **Função:** Extrai microdados de internações do SUS e aplica o filtro cirúrgico para reter exclusivamente as CID-10 iniciadas em "F" (Transtornos Mentais), calculando custos e durações de internação.
- **Saída:** csv/internacoes_saude_mental.csv

3.5. ETL 05: Benchmark Global de Depressão (OMS/GHO)

- **Arquivo:** src/scripts/etl_05_oms_who.py
- **Função:** Consome a API do Global Health Observatory, resgatando a taxa de prevalência nacional comparada a potências mundiais (EUA, Japão, Europa).
- **Saída:** csv/prevalencia_oms.csv

3.6. ETL 06: Infraestrutura Psiquiátrica (SUS RAPS)

- **Arquivo:** src/scripts/etl_06_sus_raps.py

- **Função:** Acessa os arquivos hospedados no Amazon S3 do Ministério da Saúde para mapear a oferta nacional de CAPS (Centros de Atenção Psicossocial) e Leitos Psiquiátricos Especializados.
- **Saída:** csv/infraestrutura_raps_sus.csv

3.7. ETL 07: Série Histórica de Longo Prazo (OWID)

- **Arquivo:** src/scripts/etl_07_owid_depressao.py
- **Função:** Captura a linha do tempo (1990 ao presente) da prevalência de doenças mentais via repositório do Our World In Data.
- **Saída:** csv/prevalencia_depressao_global.csv

3.8. ETL 08: Prevalência Detalhada (GBD/IHME)

- **Arquivo:** src/scripts/etl_08_gbd_ihme.py
- **Função:** Processa recortes demográficos do Global Burden of Disease, evidenciando o impacto percentual da depressão por gênero e faixa etária.
- **Saída:** csv/prevalencia_gbd_ihme.csv

4. Tratamento de Erros e Logs

Para garantir a robustez do processamento local, todos os scripts implementam tratamentos de contingência contra interrupções externas:

1. **Falha de Conexão ou Link Quebrado (HTTP 404/500):** Se as APIs do governo brasileiro ou sites de saúde caírem durante a execução, os blocos try-except evitam que o código quebre (Crash). O script ativa mecanismos de *Failover*, injetando "Dados de Contingência" (séries históricas embutidas diretamente no código) para que a estrutura relacional do banco de dados não fique vazia.
2. **Conflito de Diretórios Relativos:** Utilização de `os.path.abspath` aliado à injeção estrita de caminhos via `sys.path.insert(0, SCRIPT_DIR)`, assegurando que as importações funcionem independentemente de onde o terminal seja instanciado.
3. **Auditoria e Rastreabilidade:** Todos os robôs geram auditoria via biblioteca logging, gerando saídas visuais informando registros lidos, registros descartados, tempo de carregamento e caminhos finais dos arquivos.

5. Orquestração e Consolidação

A execução do sistema foi arquitetada para ser centralizada em dois scripts mestres:

5.1. Orquestrador Principal

- **Arquivo:** `src/scripts/run_all_etl.py`
- **Comando:** `python src/scripts/run_all_etl.py`
- **Função:** Substitui a execução manual fragmentada. Importa dinamicamente as 8 fontes ETL, aciona o método `main()` sequencialmente, mede o *runtime* de cada robô e imprime um Relatório de Execução em formato tabular no terminal.

5.2. Consolidador de Inteligência e Visualização

- **Arquivo:** `src/scripts/gerar_banco_dados.py`
- **Comando:** `python src/scripts/gerar_banco_dados.py`
- **Função:** Realiza a leitura varrendo o diretório `data/processed/csv`. Detecta novas tabelas automaticamente e consolida a base de dados nas seguintes ramificações:
 - **Banco SQLite:** `saude_mental.db`
 - **Relatório Excel:** `banco_saude_mental.xlsx`
 - **Motor de Plotagem Dinâmica (Matplotlib/Seaborn):** O algoritmo de inteligência visual identifica o tipo de dado (Série Temporal ou Dados Categóricos) mapeando nomes de colunas e gera gráficos profissionais de alto nível (.png de 300 dpi) na pasta `data/processed/plots/`, elucidando os *insights* de forma imediata.

6. Automação e Agendamento (CI/CD)

A automação do ETL do SAPA é realizada por meio de agendamento automático e integração contínua, garantindo atualização automática e contínua dos dados processados pelo sistema SAPA.

- **Frequência: Cron (Linux):** Executa diariamente o script `etl_saude_mental.py` às 06:00.
- **Gatilho Manual:** Suporte a execução sob demanda via `workflow_dispatch` para testes ou atualizações emergenciais.
- **GitHub Actions:** Automatiza a configuração do ambiente Python, instalação das dependências e execução do ETL.
- **Upload de Artefatos:** Os dados processados são armazenados automaticamente nas pastas `data/processed/` e `output/site/`.

A elaboração deste relatório contou com o apoio das ferramentas de inteligência artificial ChatGPT e Claude, utilizadas como suporte à estruturação, revisão textual e aprimoramento da redação, sob supervisão e validação dos autores.